

EVENT LOOP — ONE TURN OF LIBUV; AFTER EACH CALLBACK: PROCESS.NEXTTICK QUEUE, THEN PROMISE MICROTASKS

timers setInterval	pending callbacks deferred system errors	idle, prepare internal	poll I/O callbacks; waits here when idle	check setImmediate	close callbacks socket.on('close')
-----------------------	---	---------------------------	---	-----------------------	---------------------------------------

Order in a script: sync code → nextTick (Legacy, prefer queueMicrotask) → promises → timers/immediates. From an I/O callback setImmediate always runs before setTimeout(0); from main it is not deterministic. **Thread pool** (4 threads, UV_THREADPOOL_SIZE): most fs, dns, lookup, zlib, crypto (scrypt, pbkdf2, randomBytes). Sockets use epoll/kqueue/IOCP, not the pool. Architecture: your code → Node APIs (JS) → C++ bindings → V8 + libuv (+ Undici, libhttp) → OS.

RUNTIME, VERSIONS, INSTALL

Line	Latest	Status	EOL
26	26.10.0	Current → LTS 2026-10-28	2029-04-30
24	24.21.0	Active LTS	2028-04-30
Krypton			
22	Jod	22.23.3	Maintenance 2027-04-30
27	—	alpha 2026-10-28; every line LTS	2030-04-30

Production = an even-numbered LTS line. Install: nodejs.org installers, nvm / fnm / Volta (.nvmrc, "engines"), node:26-slim image. Corepack is no longer shipped (25+): npm i -g npnm or install it directly. Stability index: 0 Deprecated, 1 Experimental (1.1 active, 1.2 RC), 2 Stable, 3 Legacy.

CLI & REPL

```
node app.js | -e "code" | -p "expr" | -c f
node --watch app.js --watch-path=src
node --run build # script from package.json
node --env-files.env --env-file-if-exists.local
node --import ./reg.js -r ./cjs.js
node --inspect[=brk] --test --test-only
node --permission --allow-fs-read=/data
node --disable-warning=ExperimentalWarning
NODE_OPTIONS="--max-old-space-size=512"
#!/usr/bin/env node # + chmod +x
REPL: node, .help, .editor, .load, .save, .break, .clear, .exit; _last result, _error; top-level await.
process.argv, slice(2), util.parseArgs.
node.config.json via --experimental-config-file.
NODE_PATH is CommonJS only.
```

MODULES: COMMONJS VS ESM

	CommonJS	ES modules
Syntax	require(), module.exports	import/export, top-level await
Chosen by	.cjs, "type":"commonjs" (default)	.mjs, "type":"module"
Paths	__dirname, __filename	import.meta.dirname, import.meta.filename, .url, .main (24.2+)

Specifiers extension optional full extension required
import fs from 'node:fs/promises'; // node: prefix
import data from './d.json' with { type: 'json' };
const m = require('./esm.mjs'); // require(esm): Stable,
// no flag since 22.12; not with top-level await
{"type":"module", "engines":{"node":">=22"}, "exports":{"." : {"import":"./esm.js", "require":"./cjs.cjs"}, {"./sub":"./lib/sub.js"}, "imports":{"#db":"./src/db.js"}}

NPM

```
npm i | npm i pkg | npm i -D pkg | npm i -g pkg
npm ci # clean install from lockfile (CI)
npm ci --omit-dev # not --production
npm ls | outdated | update | audit |fix|
npm run build | npx tool | npm exec tool
npm init -w packages/api # workspaces
npm pack --dry-run | npm publish --provenance
^1.2.3 ≥1.2.3 <2.0.0 ~1.2.3 ≥1.2.3 <1.3.0
~0.2.3 ≥0.2.3 <0.3.0 1.2.x ≥1.2.0 <1.3.0
Commit package-lock.json. Scripts: pre/post hooks, --ignore-scripts for untrusted installs.
Publish with trusted publishers (OIDC) instead of long-lived tokens.
```

ASYN

```
const sleep = (ms) =>
  new Promise(r => setTimeout(r, ms));
import { setTimeout as wait } from
  'node:timers/promises';
import { promisify } from 'node:util';
const rd = promisify(fs.readFile);
await Promise.all([a(), b()]); // fail fast
await Promise.allSettled(xs); // never rejects
await Promise.race(xs); await Promise.any(xs);
const { promise, resolve } =
  Promise.withResolvers();
await fetch(u, { signal:
  AbortSignal.timeout(5000) });
AbortSignal.any([s1, s2]);
const als = new AsyncLocalStorage();
als.run({ reqId }, () => handler());
alsgetStore();
Callback style: error first. Never mix async with
EventEmitter handlers without captureRejections or a try/catch.
```

EVENTS

```
import { EventEmitter, once, on } from
  'node:events';
const e = new EventEmitter();
e.on('x', fn); e.once('x', fn); e.off('x', fn);
e.emit('x', 1, 2); e.prependListener('x', fn);
e.setMaxListeners(20);
const [v] = await once(e, 'ready');
for await (const [a] of on(e, 'tick', { signal
  }) {}
Emitting 'error' without a listener throws. Listeners
run synchronously in registration order. new
EventEmitter({ captureRejections: true });
EventTarget/Event are the web equivalents.
```

PROCESS & ERRORS

```
process.argv | env | cwd() | exit(1) | exitCode
process.hrtime.bigint() | memoryUsage() |
  uptime()
process.on('SIGTERM', shutdown) // SIGINT,
  SIGHUP
process.on('uncaughtException', (e, origin) => {
  log(e); process.exit(1); });
process.on('unhandledRejection', r) => { throw
  r; };
process.on('beforeExit' | 'exit', fn)
Exit codes: 0 ok, 1 uncaught error, 9 invalid
argument, 128+n killed by signal n (SIGTERM 143).
error.code: ENOENT EEXIST EACCES EADDRINUSE
ECONNREFUSED ECONNRESET ETIMEDOUT EPIPE
ERR_MODULE_NOT_FOUND. Unhandled rejections crash
the process by default. new Error('m', { cause
  }). Error.isError(x), AggregateError. Do not
  resume after uncaughtException.
```

FILES & DIRECTORIES

```
import fs from 'node:fs/promises';
await fs.readFile(p, 'utf8'); await
  fs.writeFile(p, s);
await fs.appendFile | rename | copyFile | cp(a,
  b, {recursive:true});
await fs.mkdir(d, { recursive: true });
await fs.rm(d, { recursive: true, force: true
  });
await fs.readdir(d, { withFileTypes: true,
  recursive: true });
for await (const f of fs.glob('src/**/*.js')) {}
// stable 24/22.17
await fs.stat(p) | access(p) | mkdtemp(prefix)
for await (const w of fs.watch(d, { signal })
  ) {}
const fh = await fs.open(p); for await (const l
  of fh.readlines()) {}
node:path (join resolve basename extname
  relative), URL, fileURLToPath,
  os.tmpdir() / homedir() / EOL. Avoid existsSync
  races: just try and catch ENOENT. url.parse() is
  deprecated; use new URL().
```

STREAMS

Types: Readable, Writable, Duplex, Transform; objectMode; for await (const c of readable);
import { pipeline } from 'node:stream/promises';
import { createGzip } from 'node:zlib';
await pipeline(fs.createReadStream('in.csv'),
 new Transform({ transform(c, enc, cb) {
 cb(null, c); } }),
 createGzip(), fs.createWriteStream('out.gz'),
 { signal });
class Up extends Transform { _transform(c, e,
 cb) {
 cb(null, c.toString().toUpperCase()); } }
Readable.from(['a','b']); Readable.toWeb(r);
Readable.fromWeb(w);
Backpressure rule: if write() returns false, stop
and wait for 'drain'; pipeline() does it and cleans
up on error. Web Streams: ReadableStream,
TransformStream, CompressionStream('gzip'),
response.body, zlib: gzip, brotliCompress, sync
variants block the loop.

DATA: BUFFER, SQLITE, DRIVERS

```
Buffer.from('hi') | .from(s, 'base64') |
  alloc(n)
buf.toString('hex' | 'base64url') |
  Buffer.concat([a,b])
Buffer.byteLength(s) | buf.subarray(a,b) // a
  view
import { DatabaseSync } from 'node:sqlite'; //
  RC 1.2
const db = new DatabaseSync('app.db');
db.exec('CREATE TABLE IF NOT EXISTS b(id INTEGER
  PRIMARY KEY, t TEXT)');
const ins = db.prepare('INSERT INTO b(t) VALUES
  (?)');
const { lastInsertRowid } = ins.run(title);
db.prepare('SELECT * FROM b WHERE id =
  ?').get(1);
db.prepare('SELECT * FROM b').all();
Always bind parameters, never concatenate SQL.
a.allocUnsafe is not zeroed. Drivers: pg (Poo),
  mongodb, redis, Prisma/Drizzle; one shared pool
  per process.
```

HTTP, FETCH, SOCKETS

```
import http from 'node:http';
http.createServer(async (req, res) => {
  const u = new URL(req.url, 'http://x');
  res.writeHead(200, { 'content-type':
    'application/json' });
  res.end(JSON.stringify({ ok: true }));
}).listen(3000);
const r = await fetch(url, { signal:
  AbortSignal.timeout(5e3) });
if (!r.ok) throw new Error(r.status); await
  r.json();
// SSE: content-type text/event-stream; write
"data: x\n\n"
const ws = new WebSocket('wss://host/path'); //
  client global
HTTPS: https.createServer({ key, cert });
HTTP/2: http2.createSecureServer; TCP net, UDP
dgram, dns.promises. Server limits:
headersTimeout 60 s, requestTimeout 300 s,
keepAliveTimeout 5 s, maxHeaderSize; cap body
size; server.close() then
closeIdleConnections(). Behind a proxy trust only
known proxies' X-Forwarded-*.

```

CONCURRENCY MODELS

worker_threads	child_process	cluster
Unit thread, own V8 isolate	OS process	forked Node processes
Share messages, SharedArrayBuffer, transfer	stdio, IPC	one listening port
Use CPU-bound JS for	other programs, isolation	scale HTTP over cores
Cost low	high	high, memory per worker

```
new Worker(new URL('./w.js', import.meta.url), {
  workerData });
parentPort.postMessage(r); isMainThread // in
  worker
execFile('git', ['log', '-1'], cb) // args
array, no shell
spawn(cmd, args, { stdio: 'inherit' }) |
  fork('./c.js')
if (cluster.isPrimary) cluster.fork(); else
  server.listen(3000);
Async I/O needs none of these. Never pass user
input to exec or shell: true.
```

TESTING: NODE:TEST

```
import { test, describe, mock } from
  'node:test';
import assert from 'node:assert/strict';
describe('math', () => { test('adds', async (t)
  => {
    assert.equal(1 + 1, 2);
    assert.deepEqual(a, b); await
      assert.rejects(f, /bad/);
    const m = t.mock.method(obj, 'get', () => 1);
    mock.timers.enable({ apis: ['setTimeout'] });
    mock.timers.tick(1000);
    t.assert.snapshot(value); }); });
node --test [---test-only] --watch
node --test --experimental-test-coverage
node --test --test-update-snapshots
Test runner is Stable; coverage and watch mode are
Experimental; mock, module is early development.
Test servers with fetch against listen(0) (random
  port).
```

DEBUGGING & DIAGNOSTICS

```
node --inspect-brk app.js # chrome://inspect,
  VS Code
node inspect app.js | debugger;
node --cpu-prof | --heap-prof | --prof
node --heapsnapshot=signal=SIGUSR2
v8.writeHeapSnapshot()
node --report-on-fatalerror --report-on-signal
process.report.writeReport()
node --trace-warnings --trace-uncaught --enable-
  source-maps
NODE_DEBUG=http,net node app.js
performance.mark/measure;
monitorEventLoopDelay()
diagnostics_channel.channel('x').publish(msg)
Leaks: take two heap snapshots and diff; watch
process.memoryUsage().heapUsed. Logs: pino
JSON to stdout; traces: OpenTelemetry. --max-old-
  space-size tunes V8 heap.
```

SECURITY RECIPES

```
--permission --allow-fs-read=/data --allow-fs-
  write=/out
--allow-child-process --allow-worker # Stable;
--allow-net 1.1
const key = Buffer.from(process.env.KEY_HEX,
  'hex');
crypto.randomBytes(32) | randomUUID() |
  randomInt(n)
argon2('argon2id', { message: pw, nonce: salt,
  parallelism: 4, tagLength: 32, memory: 65536,
  passes: 3 }, cb); // 24.7+
await promisify(scrypt)(pw, salt, 64);
const iv = randomBytes(12); // AES-256-GCM,
  never reuse
const c = createCipheriv('aes-256-gcm', key,
  iv);
c.getAuthTag(); d.setAuthTag(tag);
timingSafeEqual(a, b) // equal-length buffers
node -p "crypto.randomBytes(32).toString('hex')"
Checklist: lockfile + npm ci, npm audit, review install
scripts, provenance, least-privilege tokens, secrets
via --env-file node code, parameterised queries,
escaped output, hmet-style headers, rate limits,
update LTS promptly. Report vulns via nodejs.org
security reporting.
```

TYPESCRIPT

Type stripping is Stable (24.12 / 25.2); on by default
since 23.6 and 22.18. node app.ts erases types,
nothing is checked: run tsc --noEmit separately.
import type { Book } from './types.ts'; // .ts
 extension
// erasable syntax only: no enum, no namespace
with
// values, no constructor parameter properties
// not applied inside node_modules; tsconfig
 ignored
// --experimental-transform-types removed in 26
// tsconfig: "module": "nodenext",
 "target": "esnext",
 // "noEmit": true, "verbatimModuleSyntax": true,
 // "erasableSyntaxOnly": true,
 // "allowImportingTsExtensions": true
.mts/.cts pick the module system; publish compiled
 JS plus .d.ts. Lint with ESLint, format with Prettier.

PRODUCTION

```
const s = http.createServer(app).listen(3000);
let closing = false;
for (const sig of ['SIGTERM', 'SIGINT'])
  process.on(sig, () => {
    if (closing) return; closing = true;
    s.close(() => process.exit(0));
    s.closeIdleConnections();
    setTimeout(() => process.exit(1),
      10_000.unref()); });
FROM node:26-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit-dev
COPY *.
USER node
CMD ["node", "src/server.js"] # exec form:
  gets SIGTERM
node --build-sea-sea-config.json # SEA, 25.5+;
  Active dev
# ("main":"app.js", "output":"app") then
  code sign
NODE_ENV=production, health endpoints, JSON logs,
  one process per container, cluster/PM2 only without
  an orchestrator. Native add-ons: Node-API;
  WebAssembly: WebAssembly, instantiate;
  node:ffi is Experimental (26.1).
```

CHANGED SINCE OLDER TUTORIALS

Corepack no longer shipped (25+)	process.nextTick Legacy (22.7+)	url.parse() deprecated (app code, 24+)
module.register() deprecated (26); use registerHooks()	--experimental-transform-types removed (26)	Fastify 4 is EOL: use Fastify 5; Express 5 is current
dotenv → --env-file, util.parseEnv()	require(esm) works, no flag (22.12+)	shell: true with an args array deprecated (DEP0190)
ts-node → type stripping		